State

State · Order lifecycle NEW→PAID→SHIPPED with illegal guards · Build with Waqar

Today's Hook

- Today's force: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- We earn State by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Phase, Order

The Problem

- Scenario: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Constraints: solve «Order lifecycle NEW→PAID→SHIPPED with illegal guards» with State — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Success looks like: SHIPPED

Structure



State: Order lifecycle NEW→PAID→SHIPPED with illegal guar

Real-World Scenario

- Running example: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Key types: Phase, Order
- Happy-path stdout cue: SHIPPED
- Failure-path stdout cue: caught=ship from NEW

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;

import java.util.*;

public final class NaiveOrder {
  public String state="NEW";
  public void pay(){ state="PAID"; }
  public void ship(){ state="SHIPPED"; } // allows ship from NEW
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Without State, call sites absorb the wrong details.
- Failure mode to show on camera: caught=ship from NEW
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

enum Phase { NEW, PAID, SHIPPED, CANCELLED }
final class Order {
  private Phase phase=Phase.NEW;
  Phase phase(){ return phase; }
  void pay(){ if(phase!=Phase.NEW) throw new IllegalStateException("pay from "+phase); phase=Phase.PAID; }
  void ship(){ if(phase!=Phase.PAID) throw new IllegalStateException("ship from "+phase); phase=Phase.SHIPPED; }
  void cancel(){ if(phase==Phase.SHIPPED) throw new IllegalStateException("cancel shipped"); phase=Phase.CANCELLED; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== State: order lifecycle ===");
    System.out.println("--- naive (illegal ship) ---");
var n=new NaiveOrder(); n.ship(); System.out.println(n.state);
System.out.println("--- state ---");
var o=new Order(); o.pay(); o.ship(); System.out.println(o.phase());
try { new Order().ship(); } catch (IllegalStateException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== State: order lifecycle ===
--- naive (illegal ship) ---
SHIPPED
--- state ---
SHIPPED
caught=ship from NEW
```

Step-by-Step Trace

- Match stdout to stages — look for: SHIPPED
- Then the injected failure: caught=ship from NEW
- Re-run: java -cp ... 22-state ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for State
// Prefer records + sealed where they shrink the demo.
// Problem under test: Order lifecycle NEW→PAID→SHIPPED with illegal guards
```

Trade-offs

- Use State when «Order lifecycle NEW→PAID→SHIPPED with illegal guards» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind State in product language.

Related Patterns

- Name the pattern usually confused with State and separate the forces.
- Sketch the roles for «Order lifecycle NEW→PAID→SHIPPED with illegal guards» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Naive pain → State boundary → Demo stdout.
- Remember the failure cue: caught=ship from NEW
- Break a ProblemTest assertion, watch it fail, restore it.